

REPORTE DE PRÁCTICA: MONITOREO AUTOMATIZADO DE SALUD DEL SISTEMA Y ARQUITECTURA DE PLANIFICACIÓN EN GNU/LINUX

Asignatura: Sistemas Operativos

Tema: Shell Scripting, Redirección de Flujos de E/S, Interfaz Pseudo-archivos /proc y Daemon de Planificación cron

1. RESUMEN Y OBJETIVOS

El presente laboratorio examina la automatización de tareas de administración de sistemas en GNU/Linux mediante el diseño de un script en Bash (monitor.sh) y su ejecución periódica desatendida a través del daemon cron. El objetivo central es analizar cómo el sistema operativo expone métricas de rendimiento en espacio de usuario, la mecánica de manipulación de descriptores de archivo para la persistencia de historial de *logs*, y las garantías de aislamiento de privilegios dentro del modelo de control de acceso discrecional (DAC) en tareas en segundo plano.

2. REGISTRO EXPERIMENTAL SINTÉTICO

Etapa	Comandos Clave	Evento / Salida Observable
1. Preparación y Permisos	mkdir -p -v ~/salud_pc nano ~/salud_pc/monitor.sh chmod +x -v ~/salud_pc/monitor.sh	Directorio creado con modo 0775 (drwxrwxr-x). El permiso del script cambia de octal 0664 (rw-rw-r--) a 0775 (rwxrwxr-x), habilitando el bit de ejecución ()
2. Pruebas de E/S en Consola	~/salud_pc/monitor.sh cat ~/salud_pc/salud.log	Muestreo manual exitoso en el host sievert. Creación del archivo salud.log mediante concatenación (>>) registrando fecha,

		hostname, uso de CPU, RAM, Disco y Uptime.
3. Automatización vía Crontab	crontab -e crontab -l	Selección de nano como editor. Registro de la regla */2 * * * * /home/teemoo/salud_pc/monitor.sh en la tabla <i>spool</i> del usuario teemoo.
4. Mapeo Temporal Continuo	cat ~/salud_pc/salud.log	Verificación de ejecuciones desatendidas consecutivas a intervalos regulares de 120 segundos (registros capturados a las 09:27:24, 09:34:12 y 09:36:01).

3. ANÁLISIS REFLEXIVO Y JUSTIFICACIÓN TEÓRICA

3.1. Extracción de Métricas y Pseudo-Sistema de Archivos (/proc)

Las herramientas utilizadas dentro del script (top, free, df, uptime) no realizan consultas directas al hardware, sino que consumen la interfaz provista por el **Pseudo-Sistema de Archivos /proc** (*procfs*) mantenido dinámicamente en memoria RAM por el kernel Linux:

- **Muestreo de CPU y RAM:** Comandos como top y free leen estructuras del kernel reflejadas en archivos virtuales como /proc/stat y /proc/meminfo.
- **Procesamiento de Cadenas:** Mediante el operador tubería (|), la salida estándar (*STDOUT*) de estas utilidades se canaliza hacia herramientas de filtrado en espacio de usuario (grep y awk). awk procesa los tokens delimitados por espacios en memoria volátil para calcular operaciones aritméticas como la inversión del porcentaje inactivo de CPU:

$$\text{Uso CPU (\%)} = 100 - \$8$$

3.2. Semántica de Redirección y Descriptores de Archivo (>>)

- **Observación:** El script acumula bloques de texto en salud.log de forma consecutiva sin sobrescribir los registros previos.
- **Justificación a nivel de Kernel:**
Al ejecutar la estructura de agrupación { ... } >> ~/salud_pc/salud.log, la *shell* realiza una llamada al sistema open() sobre el archivo de destino especificando las siguientes

banderas combinadas:

Flags de Apertura = $O_WRONLY \mid O_CREAT \mid O_APPEND$

A diferencia del operador de truncado > (O_TRUNC), la bandera O_APPEND

fuerza a que el puntero del descriptor de archivo (FD) se ubique atómicamente al final del último bloque de datos antes de cada operación de escritura (write()). Esto garantiza la integridad del historial acumulado sin pérdida de registros anteriores.

3.3. Arquitectura del Daemon cron y Entorno de Ejecución Desatendido

- **Observación:** Las ejecuciones secundarias ocurrieron automáticamente en segundo plano sin intervención del usuario interactivo.
- **Justificación Teórica:**
 1. **Supervisión Continua:** El daemon crond despierta al inicio de cada minuto del reloj del sistema y examina los archivos del directorio `spool /var/spool/cron/crontabs/`.
 2. **Ciclo de Vida del Proceso:** Al coincidir la máscara temporal (`*/2 * * * *`), crond efectúa una bifurcación de proceso vía `fork()` y establece el ID de usuario efectivo ($EUID$) coincidente con el propietario de la tabla.
 3. **Contexto No Interactivo:** A diferencia de una sesión de terminal interactiva (TTY), las tareas invocadas por cron se ejecutan en una subshell aislada con un conjunto mínimo de variables de entorno (un `$PATH` restringido y sin variable `$DISPLAY`). Por ello, la especificación de rutas absolutas como `/home/teemoo/salud_pc/monitor.sh` resulta indispensable para evitar fallos de resolución de ejecutables.

3.4. Carga de Ejecutables (execve) y Control de Acceso Discrecional (DAC)

- **Asignación de Permisos (chmod +x):** Modifica los bits de modo del inodo del archivo en el sistema de archivos (ext4/btrfs). Cuando el usuario o el daemon invoca el script, el kernel ejecuta la llamada `execve()`.
- **Interprete de Comandos (Shebang):** El kernel lee los primeros bytes del archivo (Magic Bytes `#!`). Al detectar `#!/bin/bash`, suspende la ejecución directa del binario e invoca el intérprete de Bash pasando la ruta del script como argumento.
- **Aislamiento de Privilegios:** La tarea programada no requirió permisos de superusuario (sudo) debido a que opera dentro del dominio de seguridad del usuario ordinario teemoo.

El kernel valida en cada llamada `write()` que el $EUID$ del proceso posea permiso de

escritura (w) sobre el directorio contenedor `/home/teemoo/salud_pc/` y sobre el archivo

objetivo.

4. CONCLUSIONES

1. **Transparencia en la Extracción de Métricas:** GNU/Linux abstrae la complejidad del hardware a través de pseudo-sistemas de archivos como /proc, permitiendo a los scripts de administración recopilar métricas de salud con un impacto mínimo en el rendimiento global del sistema.
2. **Garantía de Persistencia Atómica:** La utilización del operador de redirección en modo *append* (`O_APPEND`) asegura un registro seguro e incremental, fundamental para auditorías de monitoreo preventivo.
3. **Aislamiento de Procesos en Segundo Plano:** El servicio cron proporciona una infraestructura robusta para la automatización, ejecutando procesos desatendidos bajo entornos no interactivos rigurosamente delimitados.
4. **Adherencia al Modelo DAC de Mínimo Privilegio:** La ejecución exitosa del monitoreo dentro del directorio personal demuestra que la automatización de tareas administrativas de nivel de usuario no debe ni requiere elevar privilegios a root, preservando la seguridad del sistema.